

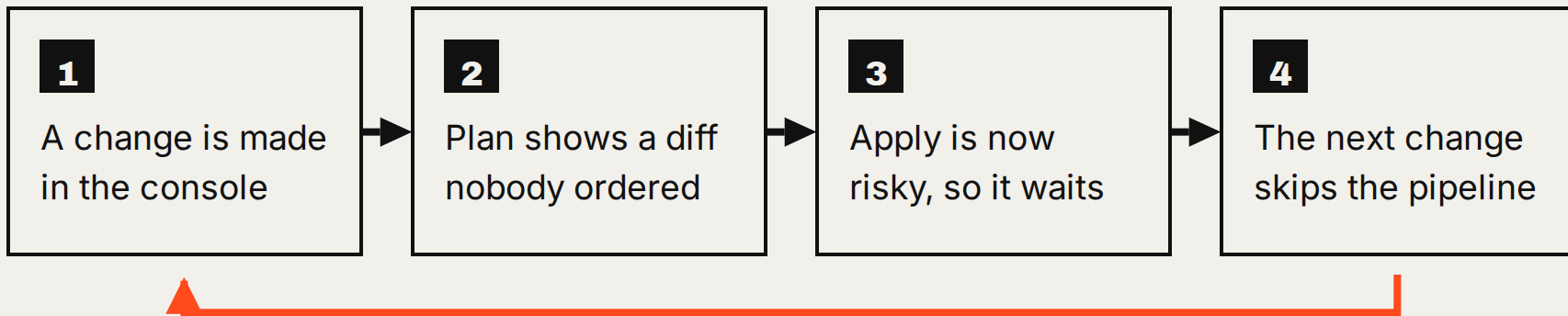
INFRASTRUCTURE

DRIFT

**THE CHANGE
SOMEONE MADE
BY HAND**

Drift is not a tooling problem. The console is still a valid write path.

The plan stops being true



Nothing here is a tooling failure. The loop closes on itself — **each lap makes the next change by hand more reasonable than the last.**

Why the console keeps winning

1

Faster during an incident

The pipeline answers in minutes. The pager answers in seconds.

2

The pipeline is blocked

Queued behind unrelated changes, waiting on a sleeping reviewer.

3

The module has no field

The abstraction is behind the provider. There is no code to write.

Every one of these is a gap in the paved path, not a failure of discipline.

Make one path the only path

1 READ-ONLY BY DEFAULT

Human roles in production carry read and describe. Write belongs to the pipeline's identity, and to nothing else.

2 BREAK-GLASS EXPIRES

Elevation is one click, time-boxed, and it writes a record: who, what, why, and when it lapsed.

3 CLOSE THE GAP

Every use of the glass files a ticket against the paved path: the slow step, the missing field.

Do the first two without the third and the drift just moves out of sight.

WHAT WE HAVE TO DECIDE

-
- 1** Who holds the break-glass, and how long before it lapses?
 - 2** Which gap do we close first: the slow step or the missing field?
 - 3** What do we do with the drift that is already in production?
-